

Spring AOP & Proxy

Aspect-Oriented Programming in the Spring Framework

Sub-Chapter 03.03

1. What is AOP?	Cross-cutting concerns: logging, security, transactions
2. AOP Terminology	Aspect, Advice, Pointcut, JoinPoint, Weaving
3. Types of Advice	@Before, @After, @AfterReturning, @AfterThrowing, @Around
4. @Aspect and @EnableAspectJAutoProxy	Activating AOP support in Spring
5. JDK Proxy vs CGLIB Proxy	Interface-based vs subclass-based proxying
6. Logging Aspect Example	End-to-end example of a reusable logging aspect

1

What is AOP?

Cross-Cutting Concerns

Aspect-Oriented Programming (AOP) separates **cross-cutting concerns** such as logging, security, and transaction management from the core business logic. Without AOP the same boilerplate code is scattered across every class that needs it.

Cross-cutting concerns solved by AOP:

- Logging - record every method entry and exit
- Security - check authorisation before executing a method
- Transactions - begin/commit/rollback a database transaction
- Caching - return a cached value instead of calling the real method
- Performance monitoring - measure how long a method takes

Without AOP (logging repeated in every method):

```
public class OrderService {
    public void placeOrder(Order o) {
        log.info("Entering placeOrder"); // repeated
        // ... business logic ...
        log.info("Exiting placeOrder"); // repeated
    }
    public void cancelOrder(Order o) {
        log.info("Entering cancelOrder"); // repeated again
        // ... business logic ...
    }
}
```

With AOP (logging centralised in one Aspect):

```
@Aspect
@Component
public class LoggingAspect {
    @Before("execution(* com.app.service.*(..))")
    public void logBefore(JoinPoint jp) {
        log.info("Entering: " + jp.getSignature().getName());
    }
}
// OrderService and CancelOrder now have ZERO logging code
```

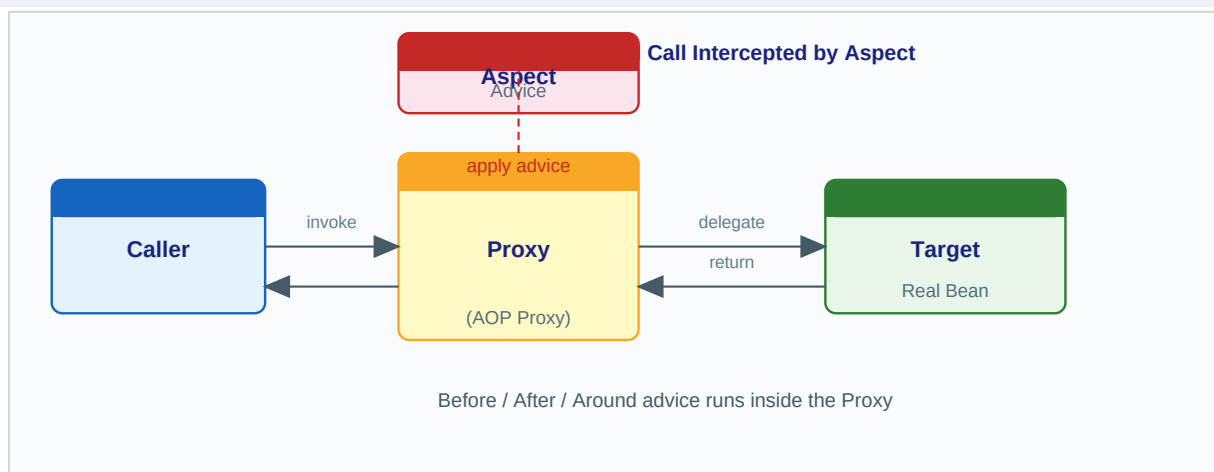


Fig 1 - AOP Concept: every method call passes through a Proxy that applies Advice

Key Rules

- AOP separates cross-cutting concerns from business logic.
- The same Aspect applies to many methods via a Pointcut expression.
- The main class stays clean - no logging/security code inside it.
- Spring AOP uses proxies at runtime (not compile-time byte-code weaving).
- Spring AOP is sufficient for most enterprise use-cases.

Q1. What problem does AOP solve?

It removes repeated cross-cutting code (logging, security, transactions) from business classes.

Q2. Name three cross-cutting concerns.

Logging, security/authorization, and transaction management.

Q3. Does Spring AOP modify the source code of the target class?

No. Spring AOP wraps the target in a proxy at runtime; the original class is unchanged.

Q4. Is Spring AOP compile-time or runtime?

Runtime - Spring creates proxy objects during the application context startup.

Q5. What is the alternative to Spring AOP for compile-time weaving?

AspectJ, which modifies byte-code at compile time and supports a wider set of join points.

2 AOP Terminology

Aspect, Advice, Pointcut, JoinPoint, Weaving

Aspect: A class that contains cross-cutting concerns (e.g. LoggingAspect). Annotated @Aspect.

Advice: The actual action at a JoinPoint (@Before, @After, @Around, etc.).

Pointcut: An expression that selects which JoinPoints the Advice applies to.

JoinPoint: A specific point in execution - in Spring AOP always a method invocation.

Weaving: The process of linking Aspects with application objects. Spring does this at runtime.

Target: The real object being proxied - the bean that contains the business logic.

Proxy: The object created by Spring to wrap the Target and apply Advice.

All five key terms in one small example:

```
@Aspect                                // Aspect
@Component
public class AuditAspect {

    // Pointcut: any method in service package
    @Pointcut("execution(* com.app.service.*(..))")
    public void serviceMethods() {}

    // Advice: runs Before each JoinPoint matched above
    @Before("serviceMethods()")
    public void audit(JoinPoint jp) { // jp = JoinPoint
        System.out.println("Auditing: " + jp.getSignature());
    }
}
// Weaving happens automatically at Spring startup
```

Key Rules

- An Aspect = @Aspect class containing one or more Advice methods.
- A JoinPoint in Spring AOP is always a method execution (not field access).
- A Pointcut is an expression - it is NOT the advice itself.
- Weaving in Spring happens at runtime via proxies (no source modification).
- The Target object has no knowledge of the Aspect applied to it.

Q1. What is an Aspect?

A modularization of a cross-cutting concern - a class annotated with @Aspect.

Q2. What is the difference between Pointcut and Advice?

A Pointcut selects where to apply; Advice is the code that actually runs at that point.

Q3. What is a JoinPoint in Spring AOP?

A specific method execution that the Advice intercepts.

Q4. When does weaving happen in Spring AOP?

At runtime, when Spring builds the application context and creates proxy beans.

Q5. What annotation marks a class as an Aspect?

@Aspect (from AspectJ). The class also needs @Component to be a Spring bean.

3 Types of Advice

@Before @After @AfterReturning @AfterThrowing @Around

Annotation	When it Runs
@Before	Runs before the method. Cannot prevent execution unless it throws.
@After	Runs after the method whether it succeeded or threw (like finally).
@AfterReturning	Runs only when the method returns normally. Can inspect the return value.
@AfterThrowing	Runs only when the method throws. Can inspect the exception.
@Around	Wraps the entire method. Must call proceed() to invoke the real method.

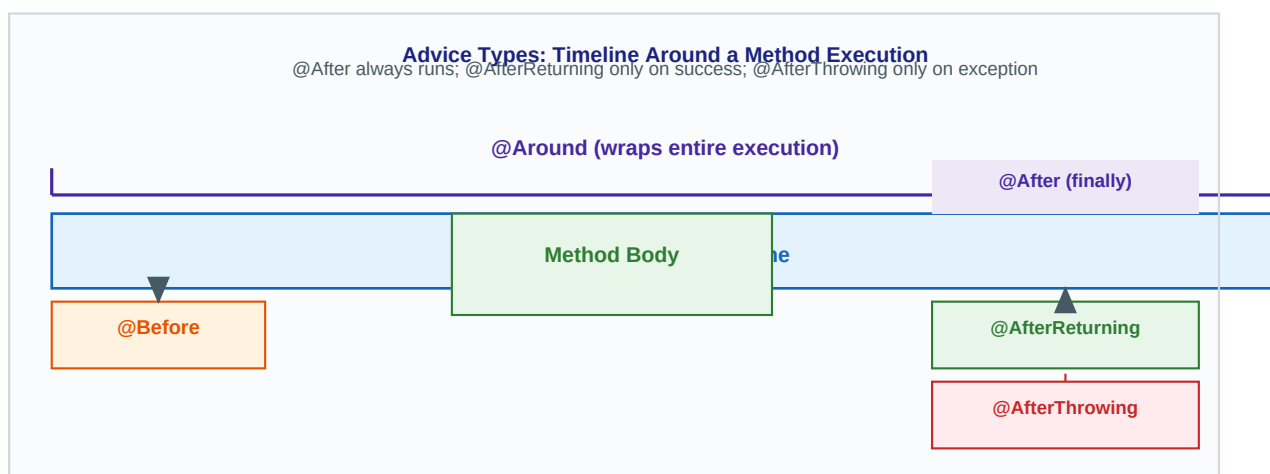


Fig 2 - Advice types mapped onto a method execution timeline

One class showing all five advice types:

```
@Aspect @Component
public class AllAdviceAspect {

    @Before("execution(* com.app.service.*.*(..))")
    public void before(JoinPoint jp) { System.out.println("BEFORE"); }

    @After("execution(* com.app.service.*.*(..))")
    public void after(JoinPoint jp) { System.out.println("AFTER"); }

    @AfterReturning(pointcut="execution(* com.app.service.*.*(..))",
        returning="result")
    public void afterOk(Object result) { System.out.println("OK: "+result); }

    @AfterThrowing(pointcut="execution(* com.app.service.*.*(..))",
        throwing="ex")
    public void afterEx(Exception ex) { System.out.println("EX: "+ex); }

    @Around("execution(* com.app.service.*.*(..))")
    public Object around(ProceedingJoinPoint pjp) throws Throwable {
        System.out.println("AROUND-start");
```

```
        Object r = pjp.proceed();
        System.out.println("AROUND-end");
        return r;
    }
}
```

Key Rules

- @Before cannot stop method execution unless it throws an exception.
- @After is like a finally block - always runs regardless of outcome.
- @AfterReturning receives the return value via the 'returning' attribute.
- @AfterThrowing receives the thrown exception via the 'throwing' attribute.
- @Around must call pjp.proceed() or the real method will never execute.

Q1. Which advice type always executes, even on exception?

@After - it behaves like a Java finally block.

Q2. How do you access the return value in @AfterReturning?

Add a 'returning' attribute with the parameter name, e.g. returning="result".

Q3. What happens if @Around advice does not call proceed()?

The real target method is never called - the proxy returns null or does nothing.

Q4. Which advice is most powerful and why?

@Around - it controls whether and how the method runs and can change arguments and return values.

Q5. Can @AfterThrowing suppress the exception?

No. To suppress or replace an exception you must use @Around.

4 @Aspect and @EnableAspectJAutoProxy

Activating Spring AOP

To use AOP in a Spring application you need two things: mark the Aspect class with **@Aspect**, and enable auto-proxy creation with **@EnableAspectJAutoProxy** on a configuration class (Spring Boot does this automatically when the AOP starter is present).

Step 1 - Add the dependency (Maven):

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-aop</artifactId>
</dependency>
```

Step 2 - Enable AOP on your configuration class:

```
@Configuration
@EnableAspectJAutoProxy          // enables @Aspect scanning
public class AppConfig { }

// In Spring Boot this is auto-configured when the AOP starter is on classpath
```

Step 3 - Write the Aspect bean:

```
@Aspect
@Component                      // must also be a Spring bean
public class TimingAspect {

    @Around("execution(* com.app.service.*(..))")
    public Object time(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed();
        long elapsed = System.currentTimeMillis() - start;
        System.out.println(pjp.getSignature() + " took " + elapsed + " ms");
        return result;
    }
}
```

Common Pointcut expression syntax:

Expression	Matches
execution(* com.app.service.*(..))	Any method in any class in com.app.service
execution(public * *(..))	Any public method anywhere
within(com.app.service.*)	Any method within the service package
@annotation(org.springframework.transaction.annotation.Transactional)	Any method annotated @Transactional
execution(* com.app..*.*(..)) && args(id,..)	Methods with first arg named id in com.app or sub-packages
Key Rules	

- **@Aspect** marks a class as an aspect - it is NOT a Spring bean by itself.
- Add **@Component** (or **@Bean**) to make the aspect discoverable by Spring.
- **@EnableAspectJAutoProxy** is required; Spring Boot adds it automatically via AOP starter.
- Pointcut expressions follow AspectJ syntax; 'execution' is the most common designator.

- Use `@Pointcut` to define a named reusable expression and reference it in advice methods.

Q1. Is `@Aspect` alone enough for Spring to pick up the aspect?

No. The aspect also needs `@Component` (or `@Bean`) to become a Spring bean.

Q2. What does `@EnableAspectJAutoProxy` do?

It tells Spring to scan for `@Aspect` beans and wrap matched beans with proxy objects.

Q3. Does Spring Boot require `@EnableAspectJAutoProxy` explicitly?

No. Adding `spring-boot-starter-aop` auto-configures it.

Q4. What is the 'execution' pointcut designator?

It matches method execution join points - the most widely used Spring AOP designator.

Q5. How do you reuse a Pointcut expression across multiple advice methods?

Define a `@Pointcut` method and reference it by its method name in advice annotations.

5 JDK Dynamic Proxy vs CGLIB Proxy

How Spring Creates Proxies

Spring AOP needs to wrap target beans in proxy objects. It uses one of two strategies depending on whether the target class implements an interface.

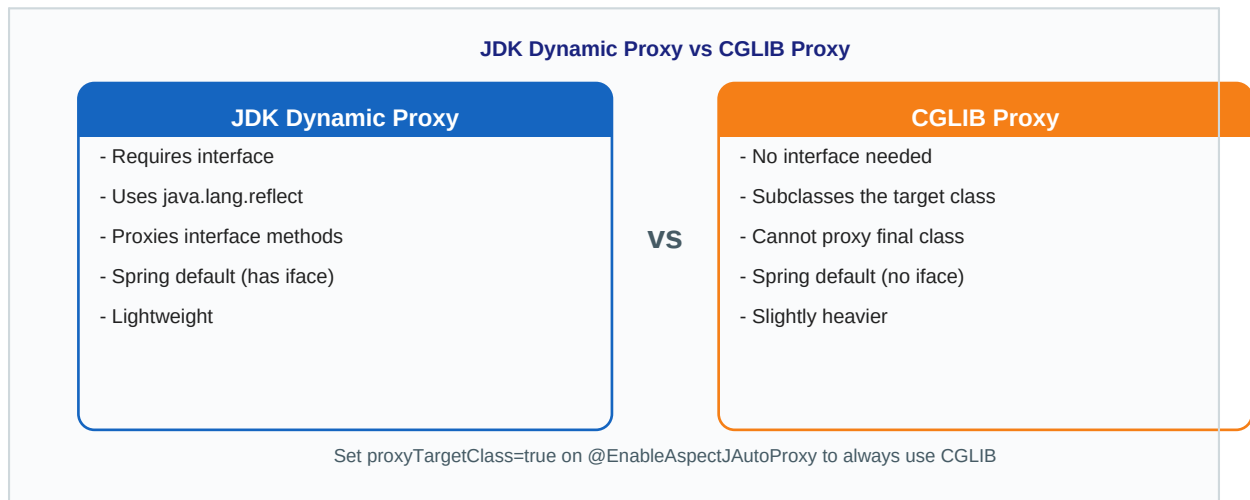


Fig 3 - JDK Dynamic Proxy (interface-based) vs CGLIB (subclass-based)

JDK Dynamic Proxy example (interface present):

```
public interface OrderService {
    void placeOrder(Order o);
}

@Service
public class OrderServiceImpl implements OrderService {
    public void placeOrder(Order o) { /* business logic */ }
}

// Spring detects the interface and creates a JDK proxy automatically.
// The proxy implements OrderService and delegates to OrderServiceImpl.
```

CGLIB Proxy example (no interface):

```
@Service
public class ProductService { // no interface
    public void addProduct(Product p) { /* business logic */ }
}

// Spring generates a subclass of ProductService at runtime.
// NOTE: class and methods must NOT be final (CGLIB cannot override final).
```

Force CGLIB even when an interface exists:

```
@Configuration
@EnableAspectJAutoProxy(proxyTargetClass = true) // forces CGLIB always
public class AppConfig { }
```

Key Rules

- JDK Dynamic Proxy requires at least one interface on the target bean.
- CGLIB generates a subclass at runtime - the target class must not be final.
- CGLIB methods must not be final; final methods cannot be advised.
- Spring chooses the proxy type automatically; you rarely need to configure it.

- proxyTargetClass=true forces CGLIB regardless of whether an interface exists.

Q1. When does Spring use JDK Dynamic Proxy?

When the target bean implements at least one interface (default Spring behaviour).

Q2. When does Spring fall back to CGLIB?

When the target bean has no interface or when proxyTargetClass=true is set.

Q3. Why can CGLIB not proxy a final class?

CGLIB creates a subclass; Java does not allow subclassing a final class.

Q4. What is the risk of injecting the concrete class type with a JDK proxy?

Spring injects a proxy that only implements the interface so casting to the concrete class will fail.

Q5. How do you always force CGLIB?

Set proxyTargetClass = true on @EnableAspectJAutoProxy.

6 Simple Logging Aspect Example

End-to-End Reusable Logging Aspect

This section assembles a complete working logging aspect that logs method entry/exit, execution time, and any exceptions - all without touching the service classes.

The Logging Aspect:

```
@Aspect
@Component
public class LoggingAspect {

    private static final Logger log =
        LoggerFactory.getLogger(LoggingAspect.class);

    @Pointcut("execution(* com.app.service.*.*(..))")
    public void serviceMethods() {}

    @Around("serviceMethods()")
    public Object logAround(ProceedingJoinPoint pjp) throws Throwable {
        String method = pjp.getSignature().toShortString();
        log.info(">> Entering: {}", method);
        long start = System.currentTimeMillis();
        try {
            Object result = pjp.proceed();
            log.info(">> Exiting: {} ({}ms)", method,
                System.currentTimeMillis() - start);
            return result;
        } catch (Throwable ex) {
            log.error("!! Exception in {}: {}", method, ex.getMessage());
            throw ex;
        }
    }
}
```

The Service class (completely clean - no logging code):

```
@Service
public class UserService {

    public User findById(Long id) {
        // just business logic - no logging needed here
        return userRepository.findById(id).orElseThrow();
    }

    public void deleteUser(Long id) {
        userRepository.deleteById(id);
    }
}
```

Sample console output when calling findById(42):

```
INFO LoggingAspect - >> Entering: UserService.findById(..)
INFO LoggingAspect - >> Exiting: UserService.findById(..) (12ms)

// If an exception occurs:
ERROR LoggingAspect - !! Exception in UserService.findById(..): User not found
```

Main application class:

```
@SpringBootApplication // @EnableAspectJAutoProxy included automatically
public class DemoApp {
    public static void main(String[] args) {
```

```
        SpringApplication.run(DemoApp.class, args);
    }
}
```

Key Rules

- One Aspect class replaces hundreds of scattered `log.info()` calls.
- Use `@Around` instead of separate `@Before/@After` to measure elapsed time.
- Re-throw the caught exception so normal exception handling still works.
- The named `@Pointcut` method makes the expression reusable across multiple advice.
- Add `@Profile('dev')` on the Aspect to activate logging only in development.

Q1. Why is `@Around` preferred for logging over separate `@Before` and `@After`?

Because `@Around` runs in one method giving access to both entry and exit for elapsed-time measurement.

Q2. What must you do after catching the exception in `@Around` advice?

Re-throw it so Spring and the caller still see the exception and handle it correctly.

Q3. How would you restrict the aspect to only dev/test environments?

Annotate the aspect class with `@Profile('dev')` so it is only registered in that Spring profile.

Q4. Can you pass method arguments into the advice?

Yes. Use `args()` in the pointcut and add matching parameters to the advice method signature.

Q5. How do you unit-test the service without the logging aspect interfering?

Instantiate the service directly without a Spring context or use `@ExtendWith(MockitoExtension)`.